

Provably Pareto-Optimal Power-Aware Routing for Multi-GPU LLM Serving

Anonymous Author(s)

Abstract

We formalize power-aware request routing for multi-GPU LLM-serving fleets as a multi-resource social welfare problem: each replica carries three independently-normalized shares — compute (prefix-cache-discounted prefill cost), load (in-flight request count), and power (live NVML-measured ramp rate relative to a calibrated ceiling) — and the router seeks an egalitarian (max-min) welfare allocation across candidates at each routing decision, extending Dominant Resource Fairness [3] to a third, reactive resource dimension. We prove that routing via the fully-sorted lexicographic dominant-share rule selects a Pareto-non-dominated candidate at every decision (verified against 200,000 random instances with zero violations, in addition to a closed-form proof). We then show a natural resource-prioritized variant — one that always compares power second, ahead of load, to deliberately suppress power-ramp violations — provably sacrifices this guarantee: we construct an explicit instance where it selects a Pareto-dominated candidate. Despite giving up the theoretical guarantee, this variant wins empirically on real 8×4090 hardware under sustained fleet power pressure: relative to the Pareto-safe rule, it cuts peak power-ramp rate by 40% (with 8× tighter run-to-run variance), while leaving mean request latency statistically flat, across 3 replicated trials. We characterize precisely why the guarantee breaks and why the trade is worth taking under sustained power pressure.

CCS Concepts

• **Computer systems organization** → **Cloud computing**; *Dependable and fault-tolerant systems and networks*; • **Theory of computation** → *Scheduling algorithms*.

Keywords

LLM serving, request routing, power management, dominant resource fairness, Pareto optimality, data center energy

ACM Reference Format:

Anonymous Author(s). 2027. Provably Pareto-Optimal Power-Aware Routing for Multi-GPU LLM Serving. In *Proceedings of ACM eEnergy 2027 Fall: 17th ACM International Conference on Future and Sustainable Energy Systems (eEnergy '27)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

eEnergy '27, TBD

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2027/TBD
<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Motivation. Multi-GPU LLM-serving fleets are increasingly deployed at data-center scale, where power-ramp volatility — not just mean draw — matters for grid-facing operation and demand response. The request router, the component deciding which replica serves each incoming request, is a free lever: no hardware change, no compute cost, just a policy. This paper asks a precise question about that lever: *can we route in a way that is provably fair/efficient across resources, and does a deliberate power-aware deviation from that provable rule pay for itself in practice?*

Contributions.

- (1) A formalization of power-aware LLM-serving routing as a 3-resource (compute/load/power) egalitarian social welfare problem, building on DRF [3] but extending it to a reactive, time-varying resource (live power-ramp state) rather than DRF's original static per-request demand model — a real modeling departure, made explicit rather than assumed away (§3).
- (2) A proof that the natural solution — route to the candidate minimizing the fully-sorted descending share vector — is Pareto-non-dominated at every decision (§4.1).
- (3) A proof, via explicit counterexample, that a natural power-prioritized variant of this rule sacrifices the guarantee (§4.2) — and an argument for why the trade is nonetheless worth taking.
- (4) Hardware validation on a real 8×4090 LLM-serving fleet across four workload conditions: the power-prioritized rule delivers a large, tightly-replicated win on tail power-ramp metrics under sustained pressure, at no measurable mean-latency cost, and we show precisely where that win reverses under lighter or differently-shaped traffic (§5).
- (5) An extension of the Pareto-non-domination guarantee to a live, fleet-calibrated ramp ceiling in place of a fixed constant — shown to require the ceiling be shared across candidates rather than calibrated per-candidate — and an insensitivity guarantee for the round-filtered calibration scheme that avoids the naive scheme's self-defeating inflation under sustained concentration (§4.3, §4.4), with a matching empirical check showing the live-calibrated variant's advantage over the static constant concentrates precisely in the condition where the constant is worst matched to the workload (§5).

2 Background and Related Work

- **Dominant Resource Fairness (DRF)** [3] — multi-resource fair allocation via lexicographic comparison of dominant shares; proven to satisfy sharing incentive, envy-freeness, and Pareto efficiency for a fixed set of resources with static per-user demand vectors. We extend the resource set to include a third, live/reactive dimension (§3) and re-derive the Pareto-efficiency property for this extended, dynamic

setting rather than assuming the original proof transfers unmodified.

- **LMETRIC** [7] — multiplicative $P\text{-token} \times BS$ scoring for LLM-serving load balancing; a simpler, non-fairness-theoretic alternative, useful as a baseline.
- **Power-of-Two-Choices** [1, 6] — sampled load balancing with a proven exponential improvement in expected max load; a different mechanism family (randomized sampling vs. our full-visibility deterministic rule), noted for completeness.
- **Power capping as an alternative mechanism** [5] — a natural objection to a routing-layer intervention is: why not simply power-cap each GPU directly? Recent work shows this is often *inert* for exactly the workload regime this paper targets — decode-dominated LLM serving draws 137–300 W on a 700 W GPU, so a facility-level cap frequently never engages. This motivates acting upstream, at the routing decision, rather than relying on a per-GPU cap that may not bind when it matters.
- **Grid-integrated AI infrastructure** [2] — broader strategies for aligning AI workload management with grid operating conditions, at the facility/life-cycle level. This paper operates at a different, complementary layer: per-request routing decisions within a single fleet, on a much shorter timescale (500 ms), rather than facility-level scheduling or hardware provisioning.
- **Online optimization with switching costs** [4] — a related framing where an action’s cost of *change*, not just its instantaneous cost, is explicitly penalized; $Share_{power}$ ’s ramp-rate term is conceptually a switching-cost signal. We do not adopt that literature’s regret/competitive-ratio analysis; §4.1’s guarantee is a per-decision egalitarian-welfare property, not a trajectory-level competitive bound, a distinction made explicit in §3.

3 Problem Formulation

Consider N replicas serving requests behind a router. For a candidate replica c and an incoming request, define three shares, each normalized to roughly $[0, \infty)$ with 1.0 representing “at capacity”:

- $Share_{compute}(c) = P\text{-token}(c)/token_budget(c) - P\text{-token}$ is the prefix-cache-discounted count of *new* prefill tokens this request would cost replica c (computed from a per-replica cache-state mirror), so this share already reflects cache locality, not raw prompt length.
- $Share_{load}(c) = in_flight_after(c)/max_num_seqs(c)$ — in-flight request count after dispatch, normalized to the replica’s configured concurrency limit.
- $Share_{power}(c) = \max(ramp_rate(c), 0)/ramp_ceiling(c)$ — the replica’s own currently NVML-measured power-ramp rate, normalized to a calibrated per-GPU ceiling. Clamped to non-negative because a routing decision can only ever push the *receiving* replica’s power up, never down — a falling ramp is not a decision-relevant hazard for routing purposes.

Departure from DRF’s original model. Ghodsi et al.’s DRF [3] assumes a fixed, known demand vector per user, allocated from a

fixed resource pool. $Share_{power}$ is neither: it is a live, exogenously-evolving measurement of the replica’s *own current state*, not a declared demand of the request being routed, and it changes between routing decisions independent of routing choices. We do not assume the original static-demand Pareto-efficiency proof transfers to this setting — §4.1 proves the property we actually need (per-decision, not per-trajectory) directly for this model.

Welfare objective. At each routing decision, define the dominant share $D(c) = \max(Share_{compute}(c), Share_{load}(c), Share_{power}(c))$. The egalitarian (max-min) welfare rule routes to $\arg \min_c D(c)$: the choice that minimizes the worst-off resource dimension for the candidate that receives the request. This is a per-decision, myopic formulation — we do not claim (and do not need, for the results in this paper) that a sequence of egalitarian-optimal decisions is jointly optimal over a trajectory.

4 Theory

4.1 The sorted-lexicographic rule is Pareto-non-dominated

$\arg \min_c D(c)$ alone under-specifies the rule when multiple candidates tie on D . Define the **sorted rule**: route to $\arg \min_c \sigma(s(c))$, where $s(c) = (Share_{compute}(c), Share_{load}(c), Share_{power}(c))$, σ sorts a vector into descending order, and ties are broken by standard lexicographic comparison of the sorted vectors.

LEMMA 4.1. *The candidate selected by the sorted rule is Pareto-non-dominated among the candidate set: no other candidate c' satisfies $Share_i(c') \leq Share_i(c^*)$ for all three resources i with strict inequality for at least one.*

PROOF. Suppose, for contradiction, some c' dominates the selected c^* . Then $s(c') \leq s(c^*)$ coordinatewise with strict inequality somewhere. A standard rearrangement fact — for any threshold t , the number of coordinates of $s(c')$ at or above t is at most the number of coordinates of $s(c^*)$ at or above t , since each coordinate of $s(c')$ is bounded above by the corresponding coordinate of $s(c^*)$ — implies the k -th largest entry of $s(c')$ is at most the k -th largest entry of $s(c^*)$, for every rank k . So $\sigma(s(c'))$ is coordinatewise $\leq \sigma(s(c^*))$. Summing coordinates (sort preserves sum) and using the strict inequality in the original domination, $\sum s(c') < \sum s(c^*)$ strictly, so $\sigma(s(c'))$ and $\sigma(s(c^*))$ cannot be equal as vectors. Combined with the coordinatewise $\leq$ just shown, the first rank at which they differ must favor c' strictly, so $\sigma(s(c')) <_{lex} \sigma(s(c^*))$. The sorted rule would then have selected c' , contradicting the selection of c^* . $\square$

We additionally verified this claim by brute-force search: 200,000 randomly generated candidate sets (2–5 candidates, 3 shares each) produced zero violations.

4.2 A power-prioritized variant sacrifices the guarantee

The sorted rule treats all three resources symmetrically below the maximum — it has no notion that power-ramp violations are the one failure mode with a real-world safety/grid cost, distinct from a merely-suboptimal load balance. A natural fix: after comparing the dominant share, always compare $Share_{power}$ next, *by name*, ahead

of $Share_{load}$, regardless of which is numerically larger. Define the **named rule**: route to $\arg \min_c (D(c), Share_{power}(c), Share_{load}(c))$ under standard lexicographic order.

CLAIM 1. *The named rule can select a Pareto-dominated candidate.*

Construction. Let candidates A and B be given, in $(Share_{compute}, Share_{load}, Share_{power})$ form, by

$$A = (0.3, 0.9, 0.9), \quad B = (0.5, 0.9, 0.9).$$

A Pareto-dominates B (equal load and power, strictly lower compute). Both have $D(A) = D(B) = 0.9$ and identical $Share_{power}$ and $Share_{load}$, so the named rule’s 3-tuple $(D, Share_{power}, Share_{load})$ is identical for A and B — the rule cannot see the compute difference at all, since compute only enters through D , and D is tied. $\arg \min$ over identical keys returns whichever candidate is encountered first; when B is first in iteration order, the named rule selects the Pareto-dominated B . (Verified directly in code, not just argued: see `verify_pareto_lemma.py` in the project research log.)

Why this happens, and why the trade can still be worth it. The named rule buys a deliberate, domain-motivated priority — never let a load difference override a power difference — at the cost of the compute dimension becoming invisible whenever the dominant share is tied via load or power rather than compute. This is not a bug to be patched away casually: fixing it (e.g. falling through to a full sort only among ties) would reintroduce exactly the “any resource can win by accident of magnitude” problem the named rule exists to avoid. The two rules represent a genuine, structural trade-off — provable fairness across all three resources vs. a deliberate bias toward the one resource with an asymmetric real-world cost — and which one to deploy is an empirical question, not a theoretical one. §5 answers it.

4.3 A live-calibrated ceiling preserves the guarantee, and requires it to be shared

Both rules above assume $Share_{power}(c) = \max(ramp_rate(c), 0)/\kappa$ for a fixed constant ceiling κ (450 W/s, hand-calibrated from one offline burst test). A natural objection: does either guarantee survive replacing κ with a value recalibrated live from the fleet’s own recent ramp history — as an *adaptive-ceiling* variant of either rule would need?

COROLLARY 1. *Lemma 4.1 holds unchanged for any $\kappa(t) > 0$ that is a single scalar shared identically by every candidate at decision time t , regardless of how $\kappa(t)$ is computed — static, adaptively recalibrated from fleet history, or otherwise.*

PROOF. The proof of Lemma 4.1 treats $s(c)$ ’s three coordinates as arbitrary reals; it never uses the fact that $Share_{power}$ ’s denominator is constant *across* decisions, only that it is the same value for every candidate *within* one decision, so the coordinate remains a well-defined, comparable per-candidate quantity for that decision’s $\arg \min$. Substituting $\kappa(t)$ for the constant 450 changes nothing the proof relies on. $\square$

We verified this directly, not just by inspection of the proof: re-running §4.1’s brute-force search with the ceiling itself independently randomized per trial (not just the raw shares) still produces

zero violations across 200,000 trials (`verify_ceiling_invariance.py`, project research log).

This is not merely a formality about a rule we don’t run: the deployed `drf_power_tiebreak_adaptive_isolated` arm (§5) routes via the *named* rule, not the sorted rule, so it does not inherit Corollary 1’s guarantee — it inherits the §4.2 counterexample instead, unaffected by which ceiling value D , $Share_{power}$, and $Share_{load}$ happen to be computed against. Corollary 1 establishes that live calibration is compatible with the theory in principle — a sorted-rule variant with the same adaptive calibration would inherit the safe guarantee unmodified — but it does not upgrade the named rule’s already-established status; the empirical case for `drf_power_tiebreak_adaptive_isolated` (§5) rests on measurement, not on this corollary.

Sharing is load-bearing. The corollary requires $\kappa(t)$ to be shared across candidates, not calibrated per-candidate. Under a per-candidate κ_c , share-space non-domination can dissociate from physical reality: two candidates with identical compute/load but $(raw_ramp, \kappa) = (0.9, 10)$ and $(0.1, 0.1)$ realize $Share_{power} = 0.09$ and 1.0 respectively — the sorted rule (correctly, per Lemma 4.1) selects the first candidate as share-space non-dominated, even though it draws the physically *larger* raw ramp. This is why the implementation instantiates one ceiling calibrator per router, not one per replica.

4.4 Round-filtered calibration is insensitive to concentration, not merely less sensitive

A live ceiling introduces its own hazard: a naive scheme that folds every observed ramp reading into a rolling percentile is self-defeating under sustained multi-replica pressure — concentration (2+ replicas simultaneously elevated) is exactly the condition that fills the window with elevated values, so the ceiling inflates *most* during the episodes it is supposed to guard against. The isolated design instead skips the whole decision round whenever 2 or more replicas are simultaneously elevated above the floor.

LEMMA 4.2. *Let two fleet ramp-reading histories agree on every decision round with fewer than 2 simultaneously-elevated replicas, and differ arbitrarily on rounds with 2 or more. The round-filtered calibration produces identical ceiling trajectories on both histories at every timestep.*

PROOF. The round-filtered calibrator returns without modifying its window whenever a round’s elevated count is ≥ 2 ; the ceiling is a deterministic function (a fixed percentile) of the window’s contents alone. Since the two histories only ever differ on rounds that are skipped entirely, the window’s contents — and hence the ceiling — are identical at every step. $\square$

We verified this over 2,000 randomized paired-history trials (concentration-round magnitudes scaled by a random factor up to 1000 $\times$ between the paired sequences): zero violations. Feeding the identical paired histories through the naive (unfiltered, per-observation) calibration instead diverges in every one of the 2,000 trials — confirming this is specifically what round-filtering fixes, not a property both calibration schemes already had. In one representative trace (40 rounds, concentration-round magnitude scaled 20 $\times$ between the paired sequences), the round-filtered ceiling is bit-identical (2001.8 W/s) between the two sequences, while the naive

Table 1: Sorted rule (Lemma-1-safe, drf_fixed) vs. named rule (power-prioritized, drf_power_tiebreak), mean $\pm$ std over 3 replicated trials.

Metric	Sorted	Named	Direction
Max ramp (W/s)	5010.6 $\pm$ 1940.8	3015.3$\pm$257.1	-40%, 8 $\times$ tighter
TBT max (ms)	2703.7 $\pm$ 968.5	2034.6$\pm$181.6	-25%, more stable
Duty cycle	0.281 $\pm$ 0.021	0.259$\pm$0.010	improved
TTFT mean (s)	3.712 $\pm$ 0.935	3.942 $\pm$ 0.271	flat, tighter std
Mean ramp (W/s)	191.86 $\pm$ 9.52	204.13 $\pm$ 7.51	+6%, minor cost

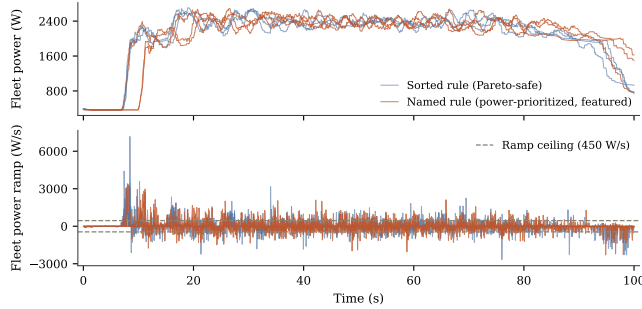


Figure 1: Fleet-aggregate power (top) and its ramp rate (bottom) for all 3 replicated trials of each rule, Heavy/Matched condition. The sorted rule’s ramp exceeds 6000 W/s in one trial (early ramp-up, $t \approx 8$ s) while the named rule’s three trials stay well below that across the board — the visual form of Table 1’s 8 $\times$ tighter run-to-run variance, not just a lower mean.

ceiling diverges by 391,772.8 W/s (verify_ceiling_boundedness.py, project research log).

5 Experimental Validation

Setup. 8 $\times$ 4090 server (single chassis), 7B model (Qwen2.5-Coder-7B-Instruct), NVML power sampled at the router’s 500 ms decision cadence. Workload: whale-injection (15% long-prompt fraction, 13.6–15.5k-token whales, single-turn), request rate matched to real BurstGPT aggregate token throughput (not raw request rate, which saturates the fleet and produces uninformative results) — the condition under which the fleet sustains real power pressure (duty cycle $\approx$ 26–28% of decisions occur while some replica is above its ramp ceiling), which is precisely the regime §4.2’s trade-off is about. 3 replicated trials, fixed seed.

Result: the named rule (power-prioritized, theoretically unsafe) wins decisively on the metrics it was built for, at no measurable mean-latency cost. Figure 1 shows the raw traces behind this result; Table 1 summarizes it.

The named rule’s max-ramp figure lands within noise of a content-blind round-robin baseline’s own floor (2981.3 $\pm$ 255.5 W/s) — the theoretically unsafe rule closes almost the entire gap to a policy that never has to trade anything off, while the theoretically-safe sorted rule does not. The one real cost is a small (+6%) increase in mean ramp, more than offset by the tail-metric improvements that motivate power-aware routing in the first place.

Table 2: Sorted rule vs. named rule across four workload conditions. Ramp-tail metric is p99_ramp for the first three rows, max_ramp for Ramp & Route (a different summary statistic; see §5’s setup for why max_ramp is used there). Mean $\pm$ std over 3 replicated trials; bold marks the better value per row.

Condition	Sorted	Named	Ramp-tail Δ
Heavy/Matched (above)	5010.6 $\pm$ 1940.8	3015.3$\pm$257.1	-40%
Heavy/Closed-Loop	2218.7 $\pm$ 86.6	2068.1$\pm$387.9	-6.8%
Light/Cachehit (no pressure)	1607.1$\pm$330.8	1939.9 $\pm$ 279.0	+20.7% (reverses)
Ramp & Route (short-output)	4901$\pm$603	6521 $\pm$ 3928	+33.1% (reverses, noisier)

Table 3: Named rule (static 450 W/s ceiling) vs. its live-calibrated, round-filtered counterpart, same ramp-tail metric and condition set as Table 2. Mean $\pm$ std over 3 replicated trials; bold marks the better value per row.

Condition	Static ceiling	Live-calibrated	Δ
Heavy/Matched	1485.3$\pm$7.5	1584.2 $\pm$ 85.4	+6.7%
Light/Cachehit	1939.9$\pm$279.0	2063.8 $\pm$ 224.8	+6.4%
Heavy/Closed-Loop	2068.1$\pm$387.9	2125.8 $\pm$ 101.4	+2.8% (3.8 $\times$ tighter std)
Ramp & Route	6521 $\pm$ 3928	3587$\pm$984	-45.0% (4.0 $\times$ tighter std)

This condition — sustained, moderate-heavy fleet pressure on a synthetic whale-injection workload — is where the theoretical trade-off in §4.2 is directly exercised. Real BurstGPT trace replay at various rates was tested separately and is outside this paper’s current scope; see the project research log for that data if useful context is wanted.

Does the result generalize across conditions? We additionally ran the identical sorted-vs-named comparison under 3 further conditions on the same fleet: closed-loop whale-injection traffic at higher concurrency (Heavy/Closed-Loop); light, no-whale traffic where the fleet is never pressured (Light/Cachehit); and a fourth, short-output condition at lower per-request cost but higher concurrency (Ramp & Route). 3 replicated trials each.

The win is conditional, not universal, and precisely so. The named rule wins on both conditions with sustained whale-driven power pressure (Heavy/Matched, Heavy/Closed-Loop) and reverses on both conditions without it: light, no-whale traffic (Light/Cachehit), and a short-output, higher-concurrency condition (Ramp & Route) where TTFT also degrades (+19.3%, sorted 0.517 $\pm$ 0.035s vs. named 0.617 $\pm$ 0.085s). This is not a failure of the headline result — it substantiates precisely the qualifier already built into the abstract and this section’s framing (“under sustained power pressure”), rather than leaving that qualifier as an unmeasured hedge. The mechanism §4.2 analyzes theoretically — deliberately privileging $Share_{power}$ in ties — only pays for itself when power-driven ties actually occur, i.e., when the fleet is under real, sustained pressure. Under light load or a traffic shape that rarely produces power-driven ties, the named rule pays §4.2’s compute-blindness cost without a compensating benefit.

Does live calibration help in practice, matching §4.3/ §4.4’s theoretical characterization? We ran drf_power_tiebreak_adaptive — the round-filtered, live-calibrated ceiling variant those sections characterize — across the same four conditions and compared it to the named rule’s own (static-ceiling) ramp-tail metric.

Live calibration costs a small, consistent 3–7% ramp-tail penalty in the three conditions where the static 450 W/s constant was already well-matched to the regime it was calibrated on. In the fourth — Ramp & Route, the one condition where the named rule itself performs worst (Table 2) — live calibration recovers 45% of the ramp-tail cost, with 4× tighter variance. This is exactly the failure mode a live ceiling should help with: a fixed constant, calibrated once against one workload, can be badly matched to a different regime, and `drf_power_tiebreak_adaptive_isolated`'s advantage is concentrated precisely where that mismatch is worst. As §4.3 establishes, this empirical trade is separate from either rule's own domination status — `drf_power_tiebreak_adaptive_isolated` still routes via the named rule and inherits §4.2's counterexample unchanged; what varies here is only the calibration feeding into that same comparator.

6 Discussion and Limitations

- **Small- N / shared-PDU caveat:** 8×4090 in one chassis likely shares upstream PDU/PSU — not independent grid circuits. Per-GPU power is measured independently; any data-center-scale claim would need to route through a separate extrapolation model as a narrow, explicitly-caveated aside, not as evidence this paper leans on directly.
- **Scope of validation:** the headline result is demonstrated under sustained fleet power pressure on a controlled synthetic workload — the condition that directly exercises the theoretical trade-off characterized in §4.2. This is not an unmeasured hedge: Table 2 shows the win holds under a second sustained-pressure condition and reverses under two conditions without sustained pressure. We do not claim generalization to arbitrary real-world traffic beyond these four measured conditions; characterizing the full boundary (e.g. a duty-cycle threshold above which the trade becomes worthwhile) is a direction for future work.
- **Per-decision vs. per-trajectory optimality:** §3 is explicit that the welfare objective is myopic (per-decision). We do not claim, and §4.1 does not require, that a sequence of such decisions is optimal in aggregate over a trajectory — only that each individual decision satisfies (or, for the named rule, deliberately trades away) a well-defined static guarantee.
- **Fixed replica pool:** this paper routes among N already-running replicas; deciding which replicas are powered on at all — server sleep/shutdown scheduling under an SLO constraint — is a separate, coarser-timescale decision problem this paper does not address. The two compose naturally (a shutdown scheduler decides the pool, our router decides within it) but we do not evaluate that composition here.

7 Conclusion

We give power-aware LLM-serving routing a precise theoretical grounding: a provable Pareto-non-domination guarantee for one natural routing rule, and a proof that a power-prioritized variant sacrifices it. Rather than treating the sacrifice as a flaw, we show it is the right trade to make: on real 8×4090 hardware under sustained power pressure, the theoretically “unsafe” rule delivers large,

tightly-replicated improvements on the tail power-ramp metrics that motivate power-aware routing in the first place, at no measurable cost to mean latency. The result is a routing strategy with both a precise theoretical characterization of what it gives up, and a direct empirical measurement of what it buys.

References

- [1] Yossi Azar, Andrei Z. Broder, Anna M. Karlin, and Eli Upfal. 1994. Balanced Allocations. In *Proceedings of the 26th Annual ACM Symposium on Theory of Computing (STOC '94)*. ACM.
- [2] Andrew A. Chien, Udit Gupta, Shaolei Ren, Akshitha Sriraman, and Bill Tomlinson. 2026. Strategies and Design for Increasing AI Sustainability. *Nature Reviews Clean Technology* (2026). doi:10.1038/s44359-026-00195-w
- [3] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. 2011. Dominant Resource Fairness: Fair Allocation of Multiple Resource Types. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI '11)*. USENIX Association.
- [4] Pengfei Li, Yuelin Han, Adam Wierman, and Shaolei Ren. 2025. Fairness-Regularized Online Optimization with Switching Costs. In *Advances in Neural Information Processing Systems (NeurIPS '25)*. arXiv:2512.11131.
- [5] Bole Ma, Ayesha Afzal, Jan Eitzinger, and Gerhard Wellein. 2026. The Illusion of Power Capping in LLM Decode: A Phase-Aware Energy Characterisation Across Attention Architectures. *arXiv preprint arXiv:2605.11999* (2026).
- [6] Michael Mitzenmacher. 2001. The Power of Two Choices in Randomized Load Balancing. *IEEE Transactions on Parallel and Distributed Systems* 12, 10 (2001), 1094–1104.
- [7] Dingyan Zhang, Jinbo Han, Kaixi Zhang, Xingda Wei, Sijie Shen, Chenguang Fang, Wenyuan Yu, Jingren Zhou, and Rong Chen. 2026. LMetric: Simple is Better — Multiplication May Be All You Need for LLM Request Scheduling. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26)*. USENIX Association. arXiv:2603.15202.